Smart Library Request Workflow in ServiceNow

A Comprehensive Implementation Guide for Academic Institutions

[INSTITUTION NAME PLACEHOLDER]

Version: 1.0

Date: March 2026

Prepared By: ServiceNow Implementation Team

Table of Contents

- 1. Executive Summary
- 2. Introduction
- 3. Problem Analysis
- 4. Project Objectives
- 5. System Architecture & Design
- 6. Stakeholder Analysis
- 7. Functional Requirements
- 8. Non-Functional Requirements
- 9. Database Design & Data Architecture
- 10. Role Configuration in ServiceNow
- 11. Table & Field Configuration
- 12. Reference Qualifier Configuration
- 13. Flow Designer – Workflow Automation
- 14. UI Policies
- 15. Access Control Lists (ACLs)
- 16. Business Rules
- 17. Notifications & Communication
- 18. Reporting & Analytics
- 19. Testing Strategy
- 20. Deployment Plan
- 21. Troubleshooting Guide
- 22. Future Enhancements & Scalability
- 23. Project Timeline & Milestones
- 24. Conclusion
- 25. Appendix

1. Executive Summary

This document outlines the technical blueprint and implementation strategy for the **Smart Library Workflow** on the ServiceNow platform. Designed for modern academic institutions, this projec digitize and automate the traditional library book borrowing process.

By leveraging ServiceNow's robust cloud capabilities, including the Now Platform, Flow Desig Reporting modules, the institution will transition from manual, paper-based tracking to a fully automa based digital system. The solution provides a seamless interface for students to request boo powerful backend for librarians to manage inventory and approvals.

Key Benefits: 40% reduction in administrative overhead, real-time inventory visibility, enhance experience, and data-driven insights into library usage trends.

2. Introduction

Academic libraries are the heart of educational institutions, yet many still rely on outdated manual l disconnected spreadsheet systems. These legacy methods result in data inconsistency, lost invento frustrating user experience for students.

ServiceNow, traditionally an IT Service Management (ITSM) tool, offers a flexible "Platform as a (PaaS) architecture ideal for building custom business applications. This project utilizes ServiceN Engine to create a custom "Smart Library" application.

Scope: The system encompasses the full lifecycle of a library book: from acquisition (catalog circulation (borrowing/returning) and reporting.

3. Problem Analysis

The current manual process is plagued by inefficiencies. Librarians spend excessive time manually
shelves, while students have no way of knowing if a book is available without physically visiting the l

Gap Analysis: Current State vs. Desired State

Parameter	Current State (Manual)	Desired State (ServiceNow)
Inventory Tracking	Physical registers / Excel sheets. Prone to errors.	Centralized database (u_book) with real-time status updates.
Request Process	In-person verbal requests or paper forms.	Digital self-service portal for students.
Approvals	Manual verification, often delayed.	Automated approval workflow routed to Librarians.
Notifications	None or manual emails.	System-triggered emails for approvals, rejections, and due dat
Visibility	No visibility into book availability.	Real-time dashboard showing "Available" books only.

4. Project Objectives

The primary objective is to deploy a functional, scalable Library Management System within 8 weeks

SMART Goals

- **Specific:** Implement a ServiceNow application with two custom tables and one automated flow.
- **Measurable:** Reduce book checkout time from 15 minutes to < 2 minutes.
- **Achievable:** Utilize out-of-the-box (OOTB) ServiceNow low-code tools.
- **Relevant:** Aligns with the institution's Digital Transformation initiative.
- **Time-bound:** Go-live scheduled for March 30, 2026.

5. System Architecture & Design

The solution is built on a **Three-Tier Architecture** hosted on the ServiceNow Cloud:

- **Data Layer:** Custom tables (u_book, u_borrow_request) extending the core platform database.
- **Logic Layer:** Business Rules, Flow Designer, ACLs, and UI Policies handling process automati
- **Presentation Layer:** Native Platform UI (Lists/Forms) and Service Portal for end-users.

> **Design Philosophy:** "Configuration over Customization." We utilize standard platform features to ensure easy upgradability and maintenance.

6. Stakeholder Analysis

Stakeholder	Role Description	Key Responsibilities
Students	End Users (Requester)	Search catalog, place borrow requests, return books on time.
Librarians	Process Owners (Fulfiller)	Manage book inventory, approve/reject requests, mark returns.
System Admin	Technical Owner	Maintain platform health, manage user access, troubleshoot errors.
IT Support	Support L1/L2	Assist users with login issues or basic navigation queries.
Management	Sponsor	Review library usage reports and fund new acquisitions.

7. Functional Requirements

ID	Requirement Description	Priority	Modu
FR-001	System must allow Librarians to create/edit Book records.	High	Invento
FR-002	System must prevent duplicate ISBN entries.	Medium	Databas
FR-003	Students must only see "Available" books when requesting.	High	UI/UX
FR-004	Borrow Requests must trigger an approval to Librarians.	High	Workflo
FR-005	System must auto-update Book status to "Issued" upon approval.	High	Automa
FR-006	Rejection of request must notify the student via email.	Medium	Notifica
FR-007	Students cannot modify requests once submitted.	Medium	Security
FR-008	Return Date field is mandatory when marking a book Returned.	Low	UI Polic

8. Non-Functional Requirements

- **Usability:** Interface must be intuitive, requiring less than 1 hour of training.
- **Performance:** Page load times must not exceed 3 seconds.
- **Scalability:** System must support up to 50,000 book records.
- **Security:** Data must be accessible only via authenticated ServiceNow sessions; roles must stric enforce Read/Write access.
- **Availability:** 99.9% uptime during academic hours (standard ServiceNow SLA).

9. Database Design & Data Architecture

9.1 Table: u_book (Book Catalog)

This table stores the inventory details.

Field Label	Field Name	Type	Reference/Choices	Mandatory
Book Title	u_title	String (200)	-	Yes
Author	u_author	String (100)	-	Yes
ISBN	u_isbn	String (40)	-	Yes (Unique)
Status	u_status	Choice	Available, Issued, Lost, Maintenance	Yes (Default: Availa
Location	u_location	String	-	No

9.2 Table: u_borrow_request (Transaction Table)

This table tracks circulation transactions.

Field Label	Field Name	Type	Reference/Choices	Mandatory
Request ID	number	Auto-Number	PREFIX: BREQ	System
Requested By	u_requested_by	Reference	sys_user	Yes
Book	u_book	Reference	u_book	Yes
Request Date	u_request_date	Date/Time	-	Yes
State	state	Choice	Requested, Approved, Rejected, Returned	Yes
Return Date	u_return_date	Date	-	No (Mandatory on R

10. Role Configuration in ServiceNow

Proper Role-Based Access Control (RBAC) is critical. We define two custom roles.

Steps to Create Roles:

1. Navigate to **User Administration > Roles**.
2. Click **New**.
3. **Librarian Role:**
   - Name: `x_univ_library.librarian`
   - Description: Full access to manage books and requests.
4. **Student Role:**
   - Name: `x_univ_library.student`
   - Description: Limited access to create requests and view available books.
5. Save records.

11. Table & Field Configuration

11.1 Creating the Book Table (u_book)

1. Navigate to **System Definition > Tables**.
2. Click **New**.
3. Label: **Book**; Name: `u_book`.
4. Check "Create access controls" and assign the `x_univ_library.librarian` role.
5. **Columns Tab:** Add fields defined in Section 9.1.
6. Click **Submit**.

11.2 Creating the Borrow Request Table (u_borrow_request)

1. Navigate to **System Definition > Tables**.
2. Click **New**.
3. Label: **Borrow Request**; Name: `u_borrow_request`.
4. Check "Auto-number" (Prefix: BREQ).
5. **Columns Tab:** Add fields defined in Section 9.2.
6. Click **Submit**.

12. Reference Qualifier Configuration

To prevent students from requesting books that are already issued, we apply a Reference Qualifi
Book field in the **Borrow Request** form.

Configuration Steps:

1. Open the **Borrow Request** form configuration (Right-click header > Configure > Dictionary).
2. Select the **u_book** column row.
3. Under the "Reference Specification" tab:
    - **Use Reference Qualifier:** Simple
    - **Condition:** `Status` IS `Available`
4. Alternatively, for Advanced usage (Script):

```
javascript: "u_status=available"
```

13. Flow Designer – Workflow Automation

We replace legacy Workflows with the modern Flow Designer.

13.1 Flow Logic

- **Trigger:** Record Created on `u_borrow_request` table.
- **Condition:** State is `Requested`.
- **Action 1 (Ask for Approval):** Send approval request to Users with role `librarian`.
- **Logic (If Approved):**
  - Update Record (Borrow Request): Set State to `Approved`.
  - Update Record (Related Book): Set Status to `Issued`.
  - Send Email: Notify Student of pickup.
- **Logic (If Rejected):**
  - Update Record (Borrow Request): Set State to `Rejected`.
  - Send Email: Notify Student of rejection reason.

14. UI Policies

14.1 Policy 1: Lock Fields on Approval

- **Short Description:** Read-only after Approval.
- **Condition:** `State` is `Approved` OR `Rejected`.
- **UI Policy Actions:** Set `u_book`, `u_requested_by`, `u_request_date` to **Read Only = True**

14.2 Policy 2: Return Date Logic

- **Short Description:** Return Date Mandatory on Return.
- **Condition:** `State` is `Returned`.
- **UI Policy Actions:** Set `u_return_date` to **Mandatory = True** and **Visible = True**.

15. Access Control Lists (ACLs)

Security rules ensure data integrity. The ACL Matrix below defines permissions.

Table	Operation	Role: Librarian	Role: Student	Condition
u_book	Read	Allow	Allow	None
u_book	Write/Create/Delete	Allow	Deny	None
u_borrow_request	Create	Deny	Allow	None
u_borrow_request	Read	Allow	Allow	User is Requester OR Role is Lib
u_borrow_request	Write	Allow	Deny	Only Librarians update status

16. Business Rules

Rule 1: Auto-set Request Date

- **Table:** u_borrow_request
- **When:** Before Insert
- **Advanced Script:**

```
(function executeRule(current, previous /*null when async*/) { // Automatically set the request date t
empty if (current.u_request_date.nil()) { current.u_request_date = new GlideDateTime(); } // Automatic
the
requested_by to the current user if (current.u_requested_by.nil()) { current.u_requested_by = gs.getUs
}
})(current, previous);
```

Rule 2: Prevent Duplicate Requests

```
(function executeRule(current, previous) { var gr = new GlideRecord('u_borrow_request');
gr.addQuery('u_requested_by', current.u_requested_by); gr.addQuery('u_book', current.u_book);
gr.addQuery('state',
'IN', 'Requested,Approved'); gr.query(); if (gr.next()) { gs.addErrorMessage('You already have an acti
request
for this book.'); current.setAbortAction(true); } })(current, previous);
```

17. Notifications & Communication

Notifications are configured in **System Notification > Email > Notifications**.

- **Notification: Book Approved**
  - **Trigger:** Record updated, State changes to Approved.
  - **Recipient:** field `u_requested_by`.
  - **Subject:** Your Book Request ${number} is Approved.
  - **Body:** "Dear ${u_requested_by.first_name}, Please collect '${u_book.u_title}' from the libra

18. Reporting & Analytics

We utilize the Report Designer to create dashboards.

Report 1: Most Borrowed Books

- **Source Type:** Table (u_borrow_request)
- **Type:** Bar Chart
- **Group By:** u_book
- **Aggregation:** Count

Report 2: Current Inventory Status

- **Source Type:** Table (u_book)
- **Type:** Donut Chart
- **Group By:** u_status
- **Visual:** Shows % of Available vs Issued books.

Report 3: Overdue Returns

- **Source:** u_borrow_request
- **Filter:** State is Approved AND Due Date < Today.
- **Type:** List View.

19. Testing Strategy

User Acceptance Testing (UAT) ensures the system meets requirements.

TC ID	Test Scenario	Steps	Expected Result
TC-001	Create Book	Librarian logs in, creates new book record.	Record saved with Status 'Available'.
TC-002	Student View	Student logs in, opens Book list.	'New' button hidden. Read-only access.
TC-003	Request Book	Student submits request for Available book.	Request created, State = 'Requested'.
TC-004	Unavailable Book	Student tries to select 'Issued' book.	Reference Qualifier hides book from list.
TC-005	Approval Flow	Librarian approves request.	Request State='Approved', Book Status='Issue

20. Deployment Plan

1. **Development:** Build application in Dev instance.
2. **Update Set Capture:** Ensure all configurations are captured in "Library App V1.0" Update Set.
3. **UAT Migration:** Commit Update Set to Test Instance. Run Test Cases (TC-001 to TC-005).
4. **Production Migration:** Commit Update Set to Prod.
5. **Data Load:** Import legacy book data via Import Sets (Excel).
6. **Sanity Check:** Verify ACLs and Flow triggers in Prod.

21. Troubleshooting Guide

Issue	Possible Cause	Solution
Flow not triggering	Trigger condition mismatch.	Check Flow properties; ensure Active flag is true.
Student cannot see Books	ACL Restriction.	Verify "Read" ACL on u_book has "Student" role.
Email not sent	Email properties disabled in Dev.	Check System Properties > Email Properties > "Email Sending Enabled"
Reference Qualifier fails	Dictionary configuration error.	Ensure "Simple" qualifier logic is correct or Script syntax is valid.

22. Future Enhancements & Scalability

Post-implementation, the system can be scaled with these features:

- **Barcode Scanning:** Integrate mobile app to scan ISBNs for instant lookup.
- **Fines Module:** Automated calculation of fines for late returns based on days overdue.
- **Integration:** Connect with the University Identity Provider (LDAP/SSO) for seamless user data s
- **Virtual Agent:** Chatbot for students to ask "Do you have Harry Potter?" and get instant answers

23. Project Timeline & Milestones

Milestone	Task Description	Duration	Owner
M1	Requirement Gathering & Sign-off	Week 1	Project Manager
M2	Database & Table Creation	Week 2	Developer
M3	Security & ACL Configuration	Week 3	Developer
M4	Flow Designer & Automation	Week 4	Developer
M5	UI Policies & Notifications	Week 5	Developer
M6	UAT & Testing	Week 6	QA Team
M7	Bug Fixes & Reporting	Week 7	Developer
M8	Go-Live & Training	Week 8	All

24. Conclusion

The "Smart Library Request Workflow" project successfully modernizes the academic library exper
implementing this robust ServiceNow architecture, the institution ensures data accuracy, reduce
labor, and significantly improves service delivery to students. This foundation allows for future scala
AI-driven library services.

25. Appendix

Glossary

- **ACL:** Access Control List – Security rules defining who can access what data.
- **Flow Designer:** ServiceNow's interface for building automated workflows.
- **Sys_id:** A unique 32-character GUID (Globally Unique Identifier) for every record.
- **Update Set:** A group of configuration changes that can be moved from Dev to Prod.

End of Document

ServiceNow Implementation Guide v1.0